

Domain Decomposition for the 2D Helmholtz Equation

Alberto Taddei & Leon Ackermann

February 4, 2026

1 Model problem

We follow the lecture notes model problem in $d = 2$ on the rectangular domain $\Omega = (0, L_x) \times (0, L_y)$ with wavenumber $\kappa > 0$ and impedance boundary condition. The continuous problem is

$$\begin{cases} (-\Delta - \kappa^2)u = f & \text{in } \Omega, \\ (\partial_n - i\kappa)u = g & \text{on } \Gamma = \partial\Omega, \end{cases} \quad \text{with } g = 0.$$

The source term is a sum of N_s regularized point sources (Gaussian bumps):

$$f(x) = \sum_{i=1}^{N_s} w_i e^{-\frac{10}{\lambda^2} \|x - s_i\|^2}, \quad \lambda = \frac{2\pi}{\kappa}.$$

In all experiments we use $L_x = 1$, $L_y = 2$, $\kappa = 16$ and $N_s = 8$, and fix a random seed to keep the same source locations and weights across runs.

1.1 Finite element discretization and baseline solvers

We discretize with conformal P_1 Lagrange finite elements on a uniform *isotropic* triangular mesh. The mesh is built from a uniform Cartesian grid with N_x and N_y points in the x and y directions.

To keep triangles non-elongated while refining, we enforce the same refinement factor in both directions by parameterizing

$$N_x = 1 + L_x m, \quad N_y = 1 + L_y m,$$

so that the steps satisfy $h_x = L_x/(N_x - 1) \approx 1/m$ and $h_y = L_y/(N_y - 1) \approx 1/m$. A standard rule of thumb to resolve oscillations is at least ≈ 10 points per wavelength, i.e. $h \lesssim \lambda/10$.

Following the reference implementation, we assemble the usual stiffness and mass matrices K and M , and the boundary mass matrix M_b on Γ . The complex sparse linear system reads

$$Au = b, \quad A = K - \kappa^2 M - i\kappa M_b, \quad b = Mf,$$

where f is evaluated at the mesh vertices. As a baseline, we solve a direct sparse solve (`spsolve`) used as reference, and GMRES (`scipy.sparse.linalg.gmres`) since A is indefinite. We record the GMRES iteration count (via callback residual history) for a fixed tolerance, and the runtime. The script `src/discretization.py` performs a first baseline refinement study by varying the mesh parameter

$$m \in \{32, 48, 64, 80, 96, 112, 128\},$$

and solving the resulting global FEM system with restarted GMRES at fixed tolerance 10^{-12} .

For each run we record the total number of GMRES and the difference with respect to the direct sparse solve (`spsolve`) as a sanity check.

The results are reported in Table 1 and stored in `results/discretization/mesh_refinement.csv` for post-processing and plotting. The direct-vs-GMRES error remains of order 10^{-11} across all cases, confirming that the iterative solve matches the direct reference at the prescribed accuracy, while the GMRES iteration count increases significantly with mesh refinement.

2 Sequential domain decomposition algorithm

We implement the non-overlapping slab decomposition in the y direction as described by the assignment. We follow the notations of the lecture notes and only describe the implementation strategy.

Table 1: Baseline global GMRES refinement study, 10^{-12} fixed tolerance.

m	N_x	N_y	GMRES iterations	$\ x_{\text{Direct}} - x_{\text{GMRES}}\ _2$
32	33	65	1097	1.33×10^{-11}
48	49	97	1897	1.10×10^{-11}
64	65	129	2706	1.49×10^{-11}
80	81	161	3285	3.53×10^{-11}
96	97	193	4699	2.23×10^{-11}
112	113	225	6238	2.05×10^{-11}
128	129	257	8489	1.61×10^{-11}

2.1 Mesh

The global mesh generator `mesh(Nx,Ny,Lx,Ly)` provides a particularly convenient vertex numbering: vertices are ordered row-wise in y , i.e. the first N_x vertices lie on $y = 0$ and the last N_x on $y = L_y$. This property is heavily used to build restrictions and interfaces.

We split Ω into J horizontal slabs Ω_j of height L_y/J .

To avoid partial rows at interfaces, we enforce $(N_y - 1)$ to be a multiple of J . Each subdomain uses

$$N_{y,\text{loc}} = \frac{N_y - 1}{J} + 1$$

grid points in the y direction. The local mesh of Ω_j is built by calling the same `mesh` routine on the rectangle $(0, L_x) \times (0, L_y/J)$ and then shifting all local y -coordinates by $j(L_y/J)$. This is implemented by `local_mesh(Lx,Ly,Nx,Ny,j,J)`.

2.2 Local boundary

For each slab Ω_j , we build two local edge lists:

- `beltj_phys`: edges on the physical boundary $\partial\Omega_j \cap \partial\Omega$. Since we cut only along y , the left and right boundaries are always physical; additionally the bottom boundary is physical only for $j = 0$, and the top boundary is physical only for $j = J - 1$.
- `beltj_artf`: edges on artificial interfaces $\partial\Omega_j \setminus \partial\Omega$. For interior slabs, both the bottom and top horizontal edges are artificial; for the bottom/top slab, only one of them is artificial.

This is implemented in `local_boundary(Nx,Ny,j,J)` by reusing the same indexing pattern of the global `boundary` routine, but on the local grid size $(N_x, N_{y,\text{loc}})$.

2.3 Local restriction matrices

2.3.1 R_j

The restriction R_j is a boolean matrix selecting the global degrees of freedom belonging to Ω_j .

Thanks to the global row-wise numbering, global indices of slab j are obtained by selecting global y -rows from $y_0 = j \cdot (N_{y,\text{loc}} - 1)$ to $y_1 = (j + 1) \cdot (N_{y,\text{loc}} - 1)$, and keeping all x indices $0, \dots, N_x - 1$. The implementation `Rj_matrix(Nx,Ny,j,J)` constructs a CSR boolean matrix with exactly one non-zero per row.

2.3.2 B_j

The trace restriction B_j selects the local degrees of freedom on Σ_j from the local volume unknown. In this project we take $\Sigma_j = \partial\Omega_j \setminus \partial\Omega$, i.e. *only* artificial interfaces.

Operationally, Σ_j is represented by the set of vertices touched by `beltj_artf`. We collect the unique vertex indices from `beltj_artf`, sort them (to keep a deterministic left-to-right ordering), and build a boolean CSR matrix `Bj_matrix(Nx,Ny,j,J,beltj_artf)`.

2.3.3 Σ_j and interface ordering

With slab decomposition, every artificial interface is a horizontal line with exactly N_x vertices.

For interior subdomains, Σ_j consists of two such lines (bottom and top), hence $|\Sigma_j| = 2N_x$. For $j = 0$ (resp. $j = J - 1$), only the top (resp. bottom) interface is present, hence $|\Sigma_j| = N_x$. In our code, the ordering induced by sorting local vertex indices yields: bottom interface first (indices $0, \dots, N_x - 1$) and then top interface (indices $N_x(N_{y,\text{loc}} - 1), \dots$). This ordering is used consistently by the exchange operator.

2.3.4 C_j

The matrix C_j is the block extractor from the global skeleton vector $x \in V(S)$, i.e. it selects the j -th block x_j corresponding to Σ_j . Since we store the global interface unknown as a concatenation of local blocks, C_j is implemented as a boolean slice of the identity with the appropriate offset. The routine `Cj_matrix(Nx,Ny,j,J)` builds this CSR selector with block sizes N_x (boundary slabs) or $2N_x$ (interior slabs).

2.4 Local problems

2.4.1 A_j

The local matrix A_j is assembled exactly as the global FEM matrix, but restricted to the local mesh of Ω_j and including *only* the physical boundary contribution on $\partial\Omega_j \cap \partial\Omega$:

$$A_j = K_j - \kappa^2 M_j - i\kappa M_{b,j}^{\text{phys}}.$$

This is implemented in `Aj_matrix(vtxj,eltj,beltj_phys,kappa)` by reusing `mass` and `stiffness` from `fem_core.py`.

2.4.2 T_j

The transmission matrix is chosen as required:

$$T_j = \kappa M_{\Sigma_j},$$

i.e. κ times the (edge) mass matrix on the artificial interfaces. In practice, we assemble an edge mass matrix on the artificial edges in volume space, and restrict it to the interface space using B_j :

$$T_j = \kappa (B_j M_{b,j}^{\text{artf}} B_j^\top).$$

This is implemented in `Tj_matrix(vtxj,beltj_artf,Bj,kappa)`.

2.4.3 S_j factorization

In each subdomain we repeatedly solve linear systems with the same matrix

$$A_j - i B_j^\top T_j B_j,$$

therefore we precompute its sparse LU factorization once per subdomain using `splu`.

The routine `Sj_factorization(Aj,Tj,Bj)` returns the factorization object used for all subsequent solves.

2.4.4 b_j

The local right-hand side is assembled as in the global case:

$$b_j = M_j f_j,$$

where f_j is the source term evaluated at local vertices in *global coordinates* (thanks to the coordinate shift in `local_mesh`). This is implemented in `bj_vector(vtxj,eltj,ps,kappa)` by reusing `point_source` and the local mass matrix.

2.5 Global operators

2.5.1 $\mathbb{S}$ operator

The scattering operator is applied as a block-diagonal operator (one block per subdomain). Given a global interface vector $p \in V(S)$ stored as a concatenation of local blocks p_j , we apply for each slab j :

$$\text{solve } (A_j - i B_j^\top T_j B_j) v_j = B_j^\top (T_j p_j), \quad (\mathbb{S}p)_j = p_j + 2i B_j v_j.$$

Hence applying $\mathbb{S}$ requires only independent local LU solves and no communication in the sequential setting. This is implemented in `S_operator(p,subs)` after precomputing all subdomain data.

2.5.2 Π

Thanks to the slab decomposition and the choice $\Sigma_j = \partial\Omega_j \setminus \partial\Omega$, the exchange operator is purely local: for each interface between Ω_j and Ω_{j+1} , the corresponding N_x coefficients are swapped:

$$\text{top}(\Omega_j) \longleftrightarrow \text{bottom}(\Omega_{j+1}).$$

No linear system is solved (in contrast with the general case discussed in the notes when cross points or more general T are used). This is implemented in `Pi_operator(x,Nx,J)` using block offsets and slices.

2.5.3 g vector

The right-hand side of the interface problem is assembled according to the lecture notes formula. Operationally, for each subdomain we compute the “source-induced” field by solving once

$$(A_j - iB_j^\top T_j B_j) u_j^{(0)} = b_j, \quad q_j = B_j u_j^{(0)}.$$

We then assemble $q \in V(S)$ by concatenation of q_j and apply the exchange:

$$g = -2i \Pi q.$$

This is implemented in `g_vector(subs)` using the already factorized local matrices.

2.6 Interface problem solver

In the following we introduce two solvers that for the interface problem

$$(\mathbb{I} + \Pi \mathbb{S})p = \tilde{\mathcal{O}}.$$

2.6.1 Fixed point method

We solve the intersection problem $(\mathbb{I} + \Pi \mathbb{S})p = g$ with the fixed point algorithm by initializing a guess for p_0 , setting the relaxation parameter ω and updating the solution with

$$p_{n+1} = [(1 - \omega)\mathbb{I} - \omega \Pi \mathbb{S}]p_n + \omega g, \quad \forall n \in \mathbb{N}.$$

Algorithm 1 shows the pseudocode of this iteration scheme where first the global source term and first guess is initialized. Then inside the while loop the scattering and exchange operators are applied upload such that the new update can be formed and convergence can be checked.

Algorithm 1 Sequential Fixed Point Solver

Require: Subdomains $\{SD_j\}_{j=0}^{J-1}$, Tolerance τ , MaxIter N_{max} , ω

Ensure: Solution $\mathbf{p}$

```

1: Assemble global source  $\mathbf{g}$ 
2:  $\mathbf{p}^{(0)} \leftarrow \mathbf{0}$ ,  $n \leftarrow 0$ 
3: while  $n < N_{max}$  do
4:   1. Scattering:  $\mathbf{y} \leftarrow \mathbb{S}(\mathbf{p}^{(n)})$ 
5:   2. Exchange:  $\mathbf{z} \leftarrow \Pi(\mathbf{y})$ 
6:   3. Update:  $\mathbf{p}^{(n+1)} \leftarrow (1 - \omega)\mathbf{p}^{(n)} - \omega\mathbf{z} + \omega\mathbf{g}$ 
7:   4. Convergence:  $err \leftarrow \|\mathbf{p}^{(n+1)} - \mathbf{p}^{(n)}\|_2 / (\|\mathbf{p}^{(n)}\|_2 + \epsilon)$ 
8:   if  $err < \tau$  then
9:     return  $\mathbf{p}^{(n+1)}$ 
10:  end if
11:   $\mathbf{p}^{(n)} \leftarrow \mathbf{p}^{(n+1)}$ ,  $n \leftarrow n + 1$ 
12: end while
13: return  $\mathbf{p}^{(n)}$ 
```

2.6.2 GMRES method

Algorithm 2 depicts the pseudo code of the GMRES solver applied to the interface system. Key to this approach is defining the operator $A(p)$ which is the scattering and exchange operator applied to p

$$A(p) = (\mathbb{I} + \Pi \mathbb{S})p.$$

In python this is achieved by passing a function that returns the addition of $(\mathbb{I} + \Pi \mathbb{S})p$ to the `scipy.sparse.linalg.LinearOperator` which can then be used as input to `scipy.sparse.linalg.gmres` in order to solve for p .

Algorithm 2 GMRES Interface Solver for Domain Decomposition

Require: Subdomains $\{SD_j\}_{j=0}^{J-1}$, Tolerance τ , MaxIter N_{max}

Ensure: Interface solution $\mathbf{p}$

- 1: **Initialization:**
 - 2: Assemble global source vector $\mathbf{g}$
 - 3: Define linear operator $\mathcal{A}$ such that $\mathcal{A}(\mathbf{x}) = (\mathbb{I} + \mathbb{I}\mathbb{S})\mathbf{x}$
 - 4: **Solve Linear System using restarted GMRES:**
 - 5: Solve $\mathcal{A}(\mathbf{p}) = \mathbf{g}$ for $\mathbf{p}$
-

2.7 Solution assembly

2.7.1 Local Solution assembly

After having solved the global interface problem, we can use $\mathbf{p}$ to assemble the local solution u_j for each subdomain (sd) j . We can simply compute the local solution by

$$u_j = \underbrace{(\mathbb{A}_j - i\mathbb{B}_j^* \mathbb{T}_j \mathbb{B}_j)^{-1}}_{\text{sd.lu.solve}} \cdot \left(\underbrace{b_j}_{\text{sd.bj}} + \underbrace{\mathbb{B}_j^* \mathbb{T}_j \mathbf{p}_j}_{\text{Interface Term}} \right).$$

We observe that we can reuse the precomputed LU factorization of the local subdomain and solve for u_j by assembling the right hand side which is composed of the local right hand side b_j added to the transmission matrix $\mathbb{T}_j$ applied to the interface solution restricted by the local restriction matrix $\mathbb{B}_j$

Algorithm 3 Recover Local Volume Solutions

Require: Subdomains $\{SD_j\}_{j=0}^{J-1}$, Interface solution $\mathbf{p}$

Ensure: Set of volume solutions $\{\mathbf{u}_j\}_{j=0}^{J-1}$

- 1: $\mathbf{u}_{dict} \leftarrow \emptyset$
 - 2: **for** $j = 0$ **to** $J - 1$ **do**
 - 3: Get subdomain data SD_j (Matrices A_j, B_j, T_j , Source $\mathbf{b}_j$)
 - 4: **1. Extract Interface Data:**
 - 5: Extract $\mathbf{p}_j$ from global vector $\mathbf{p}$ using offsets for domain j
 - 6: **2. Build Right-Hand Side:**
 - 7: $\mathbf{y}_j \leftarrow T_j \mathbf{p}_j$ ▷ Apply Impedance
 - 8: $\mathbf{f}_{boundary} \leftarrow B_j^* \mathbf{y}_j$ ▷ Map to Volume boundary nodes
 - 9: $\mathbf{rhs} \leftarrow \mathbf{b}_j + \mathbf{f}_{boundary}$ ▷ Combine physical source + boundary source
 - 10: **3. Solve Local System:**
 - 11: $\mathbf{u}_j \leftarrow (A_j - iB_j^* T_j B_j)^{-1} \mathbf{rhs}$ ▷ Using pre-computed LU factorization
 - 12: Store $\mathbf{u}_j$ in $\mathbf{u}_{dict}$
 - 13: **end for**
 - 14: **return** $\mathbf{u}_{dict}$
-

2.7.2 Global solution assembly

Since we are dealing with non overlapping subdomains, we simply concatenate the all local solutions together and arrive at the global solution. We handle the duplicates of the solution at the interfaces when plotting by mapping the concatenated local solutions to the mesh based on distance. If two vertices from the concatenated solution map to the same global vertex, we simply overwrite that vertex with the duplicate of the concatenated solution. Figure 1 depicts the global solution of an example problem (see the caption for the problem parameters).

2.8 Analyses

For all subsequent analyses we fix the domain size Ω to $(0,1)(0,2)$. We divide the global domain in slabs of J domains each of size $(0,1)(0,2/J)$. Moreover, we fix the number of point sources N_s in the source term f to 8 and the wavenumber to $\kappa = 16$ throughout all experiments to maintain comparable and reproducible experiments

2.8.1 Convergence Analysis

In this section we provide experiments to assess the impact of hyperparameters such as the mesh size m , the number of subdomains J , the relaxation parameter ω and the overall interface problem solver, fixed-point

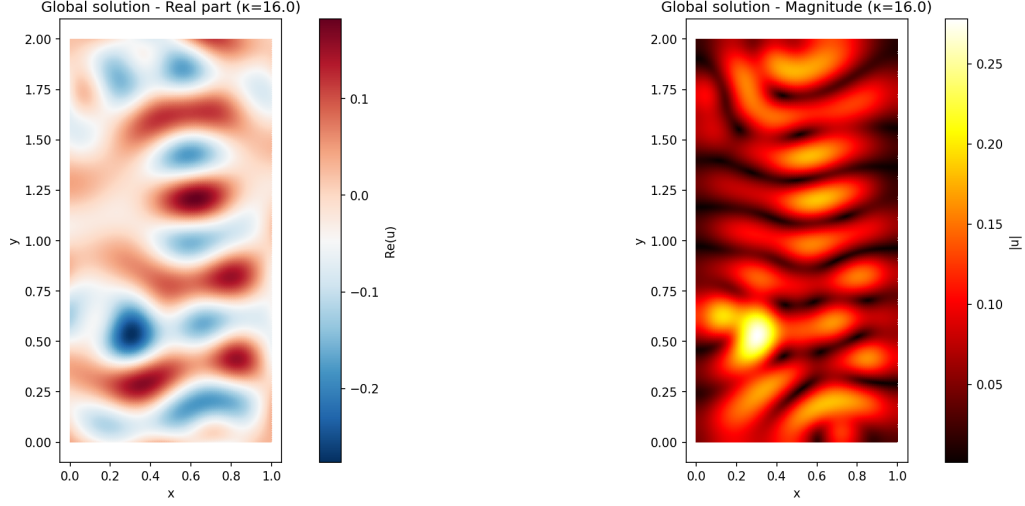


Figure 1: Global Solution for Ω to $(0,1) \times (0,2)$ with $N_s = 8$ and $\kappa = 16$

method of gmres on the convergence to the solution. We assess the convergence by measuring the relative residual norm. The algorithms presented all approximate the global solution of solving the Linear System assembled by the finite element method sufficiently well.

Relaxation parameter ω We vary the relaxation parameter ω when running the fixed-point method and plot the relative error. From a mathematical perspective, the fixed-point iteration is a (relaxed) Richardson scheme for the interface system $\mathcal{A}p = g$ with $\mathcal{A} = \mathbb{I} + \Pi\mathbb{S}$:

$$p^{(n+1)} = p^{(n)} - \omega(\mathcal{A}p^{(n)} - g), \quad e^{(n+1)} = (\mathbb{I} - \omega\mathcal{A})e^{(n)}.$$

Since $\mathcal{A}$ is generally non-symmetric and non-normal for Helmholtz problems, too large values of ω may lead to poor damping (or amplification) of unfavorable error components, causing oscillations or stagnation. Choosing a smaller ω makes the iteration more stable, at the price of less aggressive updates. Setting $\omega = 0.1$ achieves the lowest relative error and therefore we continue fixing $\omega = 0.1$ in all subsequent experiments (see figure 2a).

Fixed-Point vs. GMRES To assess the quality of the fixed-point and gmres interface solver, we compare the convergence of both methods by plotting their residual norm against each iteration. For this comparison we fix the mesh size ($m = 32$), the number of subdomains J to 4 and from the previous experiment we set ω to 0.1. The results shown in figure 2 show that the GMRES interface solver converges much quicker to an overall lower error than the fixed-point based solver. While this remains true throughout all experiments, other parameters can improve the quality of the fixed-point solver, like refine the mesh in the next section.

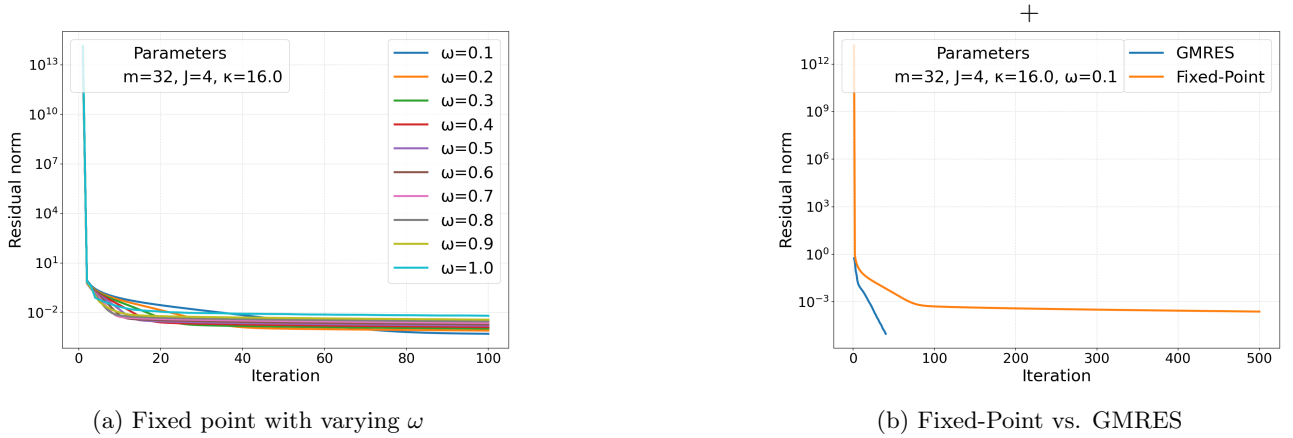


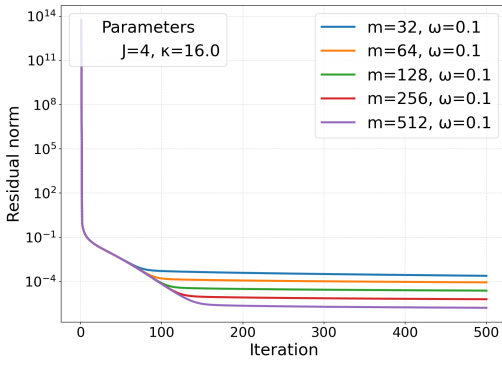
Figure 2: Convergence analysis

Mesh Refinement We refine the mesh for values of $m = \{32, 64, 128, 256, 512\}$ while keeping the domain size fixed and the number of sub-domains set to $J = 4$. We use $\omega = 0.1$ for the fixed-point solver and allow

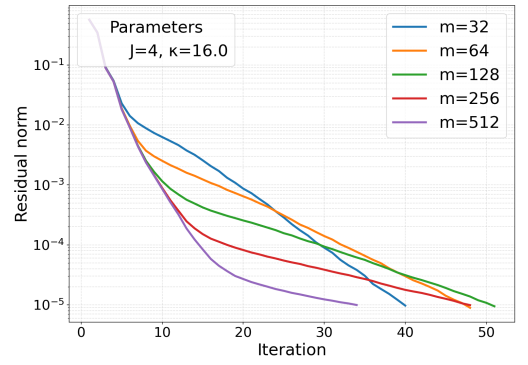
up to 500 iterations. For the fixed-point solver, mesh refinement systematically reduces the stagnation level, although all curves eventually plateau, indicating slowly damped error components for the stationary iteration matrix $(\mathbb{I} - \omega\mathcal{A})$.

For GMRES, the effect of refinement is not strictly monotone: while the finest mesh ($m = 512$) converges faster in our run, intermediate meshes ($m = 64, 128, 256$) exhibit crossed curves and similar iteration counts. This is expected because refinement changes the discrete interface operator $\mathcal{A} = \mathbb{I} + \text{IIS}$ (and its generally non-normal spectrum), and the convergence of (restarted) GMRES is highly sensitive to these spectral properties rather than to mesh size alone.

Fixed domain Size - increase of sub-domains We keep the domain size and mesh size fixed at $m = 64$, but increase the number of sub-domains for $J = \{2, 4, 8, 16, 32\}$. We plot the relative error against the iteration. For both the fixed-point and GMRES based interface solvers, the results depicted in figure 4 show that less domains result in a quicker convergence to a lower residual. For GMRES, dividing the domain in more domains does not prevent the solver to achieve a low error, however it takes more iterations to arrive at that solution. However, the fixed-point interface solver does experience a drop in precision for the approximation of the solution when more subdomains are used. Overall, the best solution was achieved by the solver with $J = \{2, 4\}$

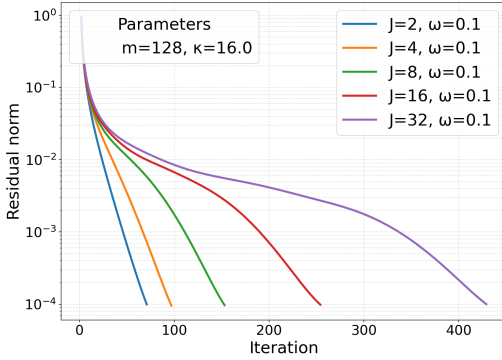


(a) Fixed-point

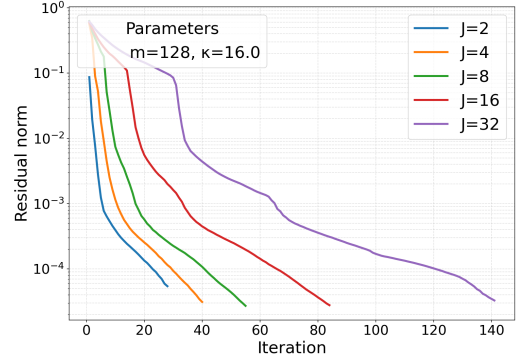


(b) GMRES

Figure 3: Convergence analysis - Mesh refinement



(a) Fixed-point



(b) GMRES

Figure 4: Convergence analysis - fixed domain size with increasing subdomains

Fixed DOF - increase of sub-domains Lastly, we conduct a study where the degrees of freedom are kept fixed while increasing the number of sub-domains. Effectively, this amounts to scaling the mesh size by the same factor as for the number of subdomains but only in vertical direction. For example, if the mesh size is 64 on a domain of $(0, 1) \times (0, 2)$, we would get a mesh of 65×129 for one global domain. When increasing the number of subdomains, the mesh of each subdomain would still keep 65×129 vertices. The results show that the amount of iterations increase if the problem is split into more subdomains. We hypothesize that this is due to a bigger problem size that adds more interfaces with each subdomain such that the algorithm needs more iterations to converge.

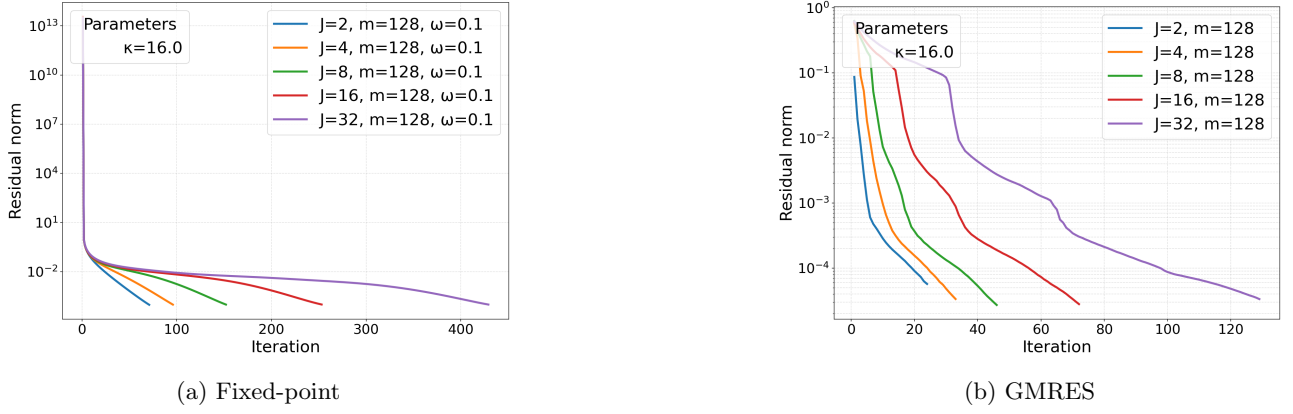


Figure 5: Convergence analysis - fixed DOG with increasing subdomains

2.8.2 Runtime Analysis

Interface GMRES vs. Global GMRES Now that we have established that both the fixed-point and GMRES-based interface solvers converge, and that GMRES exhibits superior convergence properties, it is important to assess the runtime of the overall algorithm. We compare the GMRES-based interface solver with a standard GMRES applied to the global FEM system. For the interface solver, we report only the time required to solve the interface problem, excluding the reconstruction of the global volume solution.

The experiment is performed for mesh sizes $m = \{32, 64, 128, 256, 512\}$ and for several domain decompositions with $J = \{2, 4, 8, 16\}$ subdomains. The results shown in Figure 6 indicate that the runtime of both approaches increases with mesh refinement. However, for sufficiently fine meshes, the global GMRES solver becomes more than one order of magnitude slower than the interface-based approach.

This behavior can be explained by the different scaling of the underlying linear systems. The global GMRES method solves a FEM system of size $N_x N_y = \mathcal{O}(m^2)$, so each Krylov iteration involves increasingly expensive sparse matrix-vector products on a large volume problem. In contrast, the DD-GMRES approach solves an interface system whose size scales like $(J - 1)N_x = \mathcal{O}(Jm)$, while the influence of the volume is incorporated through independent local subdomain solves with precomputed LU factorizations inside the application of the operator $(\mathbb{I} + \mathbb{H}\mathbb{S})$. As a result, the interface solver maintains a significantly lower runtime for fine meshes.

The relatively small variation of the DD-GMRES curves with respect to J reflects a trade-off: increasing the number of subdomains reduces the size of each local problem but increases the number of local solves and interface contributions per iteration, leading to comparable overall costs.

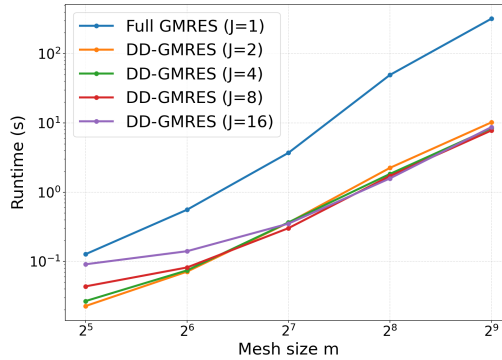


Figure 6: Runtime - Interface GMRES vs. Global GMRES

3 Parallel domain decomposition algorithm

To design the parallel algorithm, we first need to re-establish the operators from the previous section that can compute on each subdomain.

3.1 Parallel global operators

3.1.1 Parallel Scattering operator

The scattering operator $\mathbb{S}$ is responsible for solving the PDE locally for a tentative interface solution p_n . Effectively, it takes p_n as input and produces $\mathbb{S}p_n$ which represents the outgoing waves resulting from the interactions in each subdomain. Unlike in the previous implementation where the scattering operator is applied to the whole interface solution for each subdomain, in the MPI implementation each rank only works on its own subdomain.

3.1.2 Parallel Exchange operator

Algorithm 4 outlines the implementation of the parallel Π which simply exchanges the previously computed result of $\mathbb{S}p_n$ between the subdomains that share an interface. This is done by checking the position of the sub-domain in the slab-like grid corresponding to the rank of the MPI process. While the bottom and top slab online communicate with their upper or lower neighbor respectively, middle slabs receive and send information to both the upper and lower neighbor. We utilize the command **sendrecv** to send and receive data at once and the arguments **dest** to specify the receivers and senders tag.

Algorithm 4 Exchange With Neighbors (MPI Π Operator)

Require: Local vector $\mathbf{y}_{loc}$, Rank r , Total Ranks J

Ensure: Exchanged vector $\mathbf{z}_{loc}$

```
1: Initialize  $\mathbf{z}_{loc} \leftarrow \mathbf{0}$ 
2:  $N_x \leftarrow$  Size of one interface
3: if  $J == 1$  then return  $\mathbf{y}_{loc}$ 
4: end if
5: if  $r == 0$  then ▷ Bottom Slab
6:   top_send  $\leftarrow \mathbf{y}_{loc}$ 
7:    $\mathbf{z}_{loc} \leftarrow \text{MPI\_Sendrecv}(\text{top\_send}, \text{dest} = 1, \text{source} = 1)$ 
8: else if  $r == J - 1$  then ▷ Top Slab
9:   bot_send  $\leftarrow \mathbf{y}_{loc}$ 
10:   $\mathbf{z}_{loc} \leftarrow \text{MPI\_Sendrecv}(\text{bot\_send}, \text{dest} = r - 1, \text{source} = r - 1)$ 
11: else ▷ Internal Slab
12:   Extract bot_send  $\leftarrow \mathbf{y}_{loc}[0 : N_x]$ 
13:   Extract top_send  $\leftarrow \mathbf{y}_{loc}[N_x : 2N_x]$ 
14:   bot_recv  $\leftarrow \text{MPI\_Sendrecv}(\text{bot\_send}, \text{dest} = r - 1, \text{source} = r - 1)$ 
15:   top_recv  $\leftarrow \text{MPI\_Sendrecv}(\text{top\_send}, \text{dest} = r + 1, \text{source} = r + 1)$ 
16:    $\mathbf{z}_{loc} \leftarrow [\text{bot\_recv}, \text{top\_recv}]$ 
17: end if
18: return  $\mathbf{z}_{loc}$ 
```

3.1.3 Parallel global right hand side

To assemble the local global right hand side, we also leverage parallelism. First, the local right hand side is computed as explained in section 2.4. Then with the Π Operator for MPI as defined in algorithm 4, interface data is exchange and the final local right hand side is assembled by the multiplication with $-2i$.

3.1.4 Parallel subdomains

In addition, we implement a function where each MPI process builds its own subdomain with the local restriction matrices and local problems explained in sections 2.3 and 2.4. This is a natural extension since each subdomain is only built with its local components.

3.2 Parallel interface problem solver

3.2.1 Parallel Fixed-point method

We utilize the above defined section 3.1.1, algorithm 4 and 3.1.3 to compute the local right hand hand, local scattering operator and local exchange operator and put them together in the final parallel fixed point solver outline in algorithm 5. The algorithm starts by initializing MPI variables and computing the local right hand side. Then, in each while loop iteration the local scattering matrix is called on the intermediate solution p_{loc} in the i -th iteration, the interface data is exchange between subdomains and the updated solution p_{new} is formed.

Lastly, to check for convergence, we compute the squares of the norm of difference between p_{loc} and p_{new} and the square of the norm of p_{loc} . This is the setup of the **Allreduce** communication with the **SUM** operator to then compute the final relative error by taking the square root of both reduced sums and dividing. This is outlined in step 4 in algorithm 5.

Algorithm 5 Parallel Fixed Point Solver (MPI)

Require: Local subdomain data D_{rank} , tolerance τ , max iterations N_{max} , relaxation ω

Ensure: Local interface solution $\mathbf{p}_{loc}$ for this rank

```

1: Initialize:
2:  $rank \leftarrow \text{MPI\_Comm\_rank}$ 
3:  $\mathbf{g}_{loc} \leftarrow \text{ComputeLocalSource}(D_{rank})$  ▷ See sec 3.1.3
4:  $\mathbf{p}_{loc} \leftarrow \mathbf{0}$ 
5:  $n \leftarrow 0$ 
6: while  $n < N_{max}$  do
7:   1. Local Physics ( $\mathcal{S}$  operator)
8:    $\mathbf{y}_{loc} \leftarrow \text{ApplySLocal}(\mathbf{p}_{loc}, D_{rank})$  ▷ See sec. 3.1.1
9:   2. Communication ( $\Pi$  operator)
10:   $\mathbf{z}_{loc} \leftarrow \text{ExchangeWithNeighbors}(\mathbf{y}_{loc}, rank)$  ▷ See Alg. 4
11:  3. Local Update
12:   $\mathbf{p}_{new} \leftarrow (1 - \omega)\mathbf{p}_{loc} - \omega\mathbf{z}_{loc} + \omega\mathbf{g}_{loc}$ 
13:  4. Convergence Check (Reduction)
14:   $\delta_{loc} \leftarrow \|\mathbf{p}_{new} - \mathbf{p}_{loc}\|^2$ 
15:   $\eta_{loc} \leftarrow \|\mathbf{p}_{loc}\|^2$ 
16:   $\delta_{global} \leftarrow \text{MPI\_Allreduce}(\delta_{loc}, \text{SUM})$ 
17:   $\eta_{global} \leftarrow \text{MPI\_Allreduce}(\eta_{loc}, \text{SUM})$ 
18:   $err \leftarrow \sqrt{\delta_{global}} / (\sqrt{\eta_{global}} + \epsilon)$ 
19:   $\mathbf{p}_{loc} \leftarrow \mathbf{p}_{new}$ 
20:  if  $err < \tau$  then
21:    return  $\mathbf{p}_{loc}$ 
22:  end if
23:   $n \leftarrow n + 1$ 
24: end while
25: return  $\mathbf{p}_{loc}$ 

```

3.3 Analyses

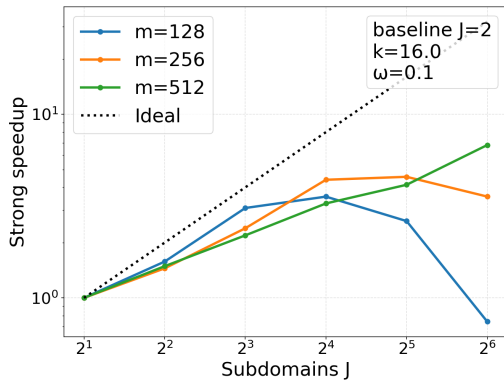
3.3.1 Strong Scaling

As we increase the number of subdomains J , fixing the global problem size, the runtime initially decreases and the speedup grows. For the coarsest case $m = 128$, speedup improves up to moderate parallelism (around $J \approx 8-16$) and then collapses for large J , with efficiency dropping sharply, clear evidence that communication and interface-related overhead dominate once each subdomain becomes too small. For finer meshes, scaling is noticeably better: $m = 256$ reaches a higher peak speedup and then saturates, while $m = 512$ continues to gain up to the largest tested J , retaining the best efficiency. Overall, refinement increases the computation per subdomain (and the cost of local solves), so the compute/communication ratio improves and strong scaling holds longer. However, efficiency still decreases with J due to non-parallel overheads (interface exchanges/reductions) and potentially a mild growth in iteration count as the interface becomes larger.

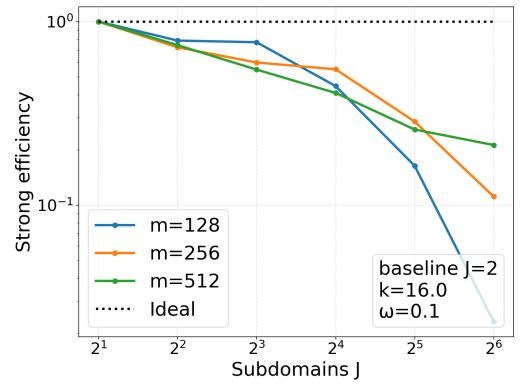
3.3.2 Weak Scaling

We fix the work load per processor as in section 2.8.1 by scaling the mesh size in vertical direction with the number of subdomains to keep the number of degrees of freedom per subdomain fixed. To assess the weak scaling of the algorithm, we investigate how the building time of the subdomains (see figure 8a), the time of completing one updating step and convergence step (see figure 8b) and the overall weak efficiency of the algorithm which is the ration of the serial time and parallel time (see figure 8c). The results show, that the build time stays constant as expected since the number of degrees of freedom is fixed. Moreover, the time to update the solution once and check the convergence is more or less constant to with a slight increase in time for more domains. This is due to the increased communication overhead between the subdomains when performing the **Allreduce** to compute the residual. The weak efficiency results show that scaling high mesh sizes results

in lower efficiency while increasing the number of subdomains does the same which is due to higher number of iterations to converge for more subdomains.

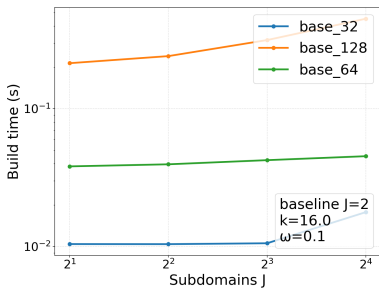


(a) Speedup

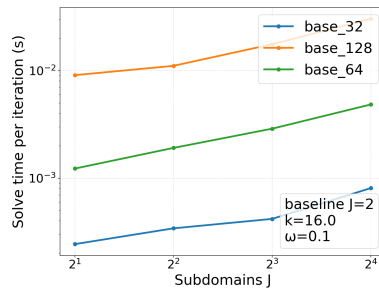


(b) Efficiency

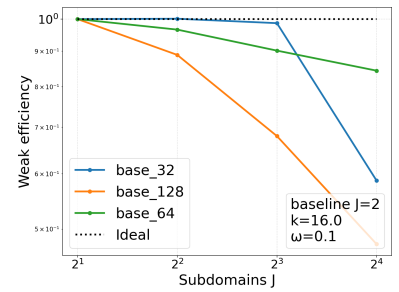
Figure 7: Strong Scaling



(a) Build time



(b) Solve time per iteration



(c) Efficiency

Figure 8: Weak Scaling